System Workflow

What the system is supposed to be, why it exists, and what it should solve - written as one typed workflow, with every section reachable from the master README.

Bottom line first. This is not a chatbot and not a folder of files. It is a judged engineering system: one shared mind map that holds what has been proven, three stores each doing one job, and a workflow where nothing enters the map without a run, a judge, and a receipt.

If a claim has no judged atom, no receipt, and no run in telemetry, it is not part of the system - however good it sounds.

> **The lock - 1 November 2026**
> Tensor KV is nested in the multi mind map. That store is **SurrealDB**.
> RAG is **Qdrant**, for memories and facts with provenance.
> Telemetry is **HyperDX**.
> Three separate stores, by job. Do not collapse them into one.

```
MASTER DOCUMENT   README.md
PUBLIC NAME       Ares - SpecOps by AtomaLabs (MadMind is the in-house reference name only)
HARNESS          RG-AH is the sole harness
STATUS           Final-run specification - stores first, proof on hardware
```

Contents - paths from the master README

Read `README.md` first. It is the master entry point. Each row below is one section of this workflow, its anchor in the master README, and the section file that holds the full text. Follow the path - do not search the repo by memory.

§	SECTION	MASTER README PATH
1	What the system is supposed to be	`README.md#1-what-the-system-is` `sections/01-system-definition.md`
2	Why it exists	`README.md#2-why-it-exists` `sections/02-why.md`
3	What it should solve - and what it should not	`README.md#3-what-it-should-solve` `sections/03-problems-solved.md`
4	Store architecture lock	`README.md#4-store-architecture` `sections/04-stores.md`
5	Execution is not storage	`README.md#5-execution-vs-storage` `sections/05-execution-plane.md`
6	The typed workflow, step by step	`README.md#6-typed-workflow` `sections/06-workflow.md`
7	DAG capability tree	`README.md#7-dag-capability-tree` `sections/07-capability-tree.md`
8	Gates, judges, and evidence	`README.md#8-gates-and-evidence` `sections/08-gates.md`
9	Build order for the final run	`README.md#9-build-order` `sections/09-build-order.md`
A	Types glossary	`README.md#appendix-a-types` `sections/appendix-a-types.md`

How to use this document: start at §1 for the definition, §2-§3 for the reason, §6 for the work itself. §4, §5, §7 and §8 are constraints - they do not bend to make a run pass.

Systems checklist added: §10-§15 on the next page - the full forward pass, TreeSitter, capability ranking, distillation, TPE, and active inference. Read §1-§9 first; §10-§15 is how a run actually moves through the machine.

Contents - systems checklist (§10-§15)

The whole thing, at systems level. Each section below is a checklist: an item is done only when its box can be ticked from a receipt, a run, or code on disk - not from intent.

§	SECTION	MASTER README PATH
10	Full forward pass - the whole pipeline, in order	`README.md#10-full-forward-pass` `sections/10-forward-pass.md`
11	Where TreeSitter sits	`README.md#11-treesitter` `sections/11-treesitter.md`
12	Capabilities - how they form and how they rank	`README.md#12-capabilities-and-ranking` `sections/12-capabilities-ranking.md`
13	Distillation - removing the weights to the map	`README.md#13-distillation` `sections/13-distillation.md`
14	TPE, toroidal scoring, and thermodynamic dissipation	`README.md#14-tpe-and-dissipation` `sections/14-tpe-dissipation.md`
15	Active inference at runtime - build only what is needed	`README.md#15-active-inference-runtime` `sections/15-active-inference.md`

How status is marked in §10-§15

- **`LOCKED`** - decided by ruling. Build to it, do not re-litigate it.
- **`VERIFIED (narrow)`** - proven, but only in the narrow form named. Do not widen the claim.
- **`TARGET`** - the design intent. Not done until the checklist item is ticked from evidence.
- **`GATE`** - must pass on hardware before anything that depends on it ships.

> **Read this first:** much of the old forward pass was unproven end to end. §10-§15 does not pretend otherwise. It turns the whole design into boxes to tick, in order, so the next pass is proven as it is built - not claimed after.

1 What the system is supposed to be

A shared, judged engineering mind that compounds with use - not a pile of separate tools, files, and claims.

One system, in one sentence

A mission comes in, is turned into a typed packet, is worked by a lane through tools, skills, and models, is judged against a pre-set bar, and - only if it passes - becomes one atom in the mind map, with a receipt that anyone can check.

Its parts

- **The multi mind map** - the single graph authority. Capabilities, atoms, edges, and the tensor key-value store nested inside it. Held in SurrealDB.
- **The nested store (N3)** - hooks, skills, tools, plugins, and Model Context Protocol servers, organised by family, plus the memory database and the research library. Order inside the store: TreeSitter, then Top MCP, then Plugins, then Skills.md, then Hooks and Tooling. Hooks, tails, and the port run before the chat completion step.
- **The harness (RG-AH)** - the sole harness. It runs missions, routes lanes, calls judges, and writes receipts. Other harnesses do not define how this system runs.
- **Lanes** - builder lanes working from one set of weights' worth of intent, specialised by task. A lane is who ran the work, not what the system knows.
- **The app surface** - Ares - SpecOps, the desktop app. The product is the engineering app with its swarm and live telemetry. Supporting runtimes are proof of ability, not the product.
- **The 12 basins** - groupings in the knowledge map. Basins are how knowledge is organised, not 12 separate agents or specialists.

> **What it is not:** not a single database doing every job, not a retrieval trick standing in for reasoning, and not a diagram of capabilities nobody has run. Stored trajectories cannot answer a problem nobody has solved before - the system has to derive, not just resemble.

2 Why it exists

Because the work outgrew loose files - and loose files quietly stopped being proof of anything.

What went wrong, plainly

- **Stores that were not databases.** Maps, trajectories, and libraries lived in JSONL files and segment piles. No transactions, no real graph traversal, no single authority to check against.
- **Trajectories put together on top of that.** Every trajectory inherited the looseness of the store under it, so runs could not be assembled or replayed reliably.
- **Claims built ahead of proof.** Much of the forward pass was never proven end to end. Bench kernels passed and map scoring passed, but the forward pass itself was unchecked - and other claims were stacked on top of it anyway.
- **Material scattered everywhere.** Duplicate stores, uncommitted layers, and work spread across folders, so nobody could say in one place what was true, what was pending, and what was withdrawn.

The lesson that sets the order

Starting with the right databases first would have avoided a lot of this. Store first, then map, then trajectories, then claims - in that order. Doing it backwards is how weeks go to patching around a store that cannot hold the thing it is supposed to prove.

What still stands

The basics are workable and do not depend on the unproven parts: tools, skills, runbooks, hooks, and the working editor setup built on 1 November. This workflow keeps those, and rebuilds the map and stores underneath them properly.

3 What it should solve

PROBLEM	WHAT SOLVED LOOKS LIKE
No single truth for the map	One graph authority in SurrealDB. One place to ask what the system knows, and one place to correct it.
Facts without provenance	Every retrieved memory or fact in Qdrant carries where it came from, so a capability can show why it claims what it claims.
Runs nobody can observe	Every run streams to HyperDX and leaves a live status, a log, and a why-receipt on failure: what ran, the command, the exit, the last output, the expected result, the evidenced cause, and the unblocker.
Unproven capabilities in the tree	A capability appears only with a judged atom, a receipt, and a real run behind it. No receipt, no node.
Context and coherence lost mid-run	N3 acts as the persistent anchor outside the session - the store a run can be pulled back to and measured against when context would otherwise drift.
Quality traded for speed	Slow and done once, with gates locked, beats doing it ten times with a different failure each time. Quality over speed is a rule, not a preference.

What it should not solve - non-goals

- It does not replace the execution plane with a database, or the database with a model.
- It does not convert regular retrieval to a different geometry on a hunch. Regular BM25 plus cosine retrieval stays.
- It does not ship on synthetic or simulated runs. Ever.
- It is not a website project and not a marketing diagram.

4 Store architecture lock

Three stores, three jobs. This section is a lock, not a proposal.

STORE	JOB	HOLDS
SurrealDB	The mind graph	The multi mind map: nodes, edges, atoms, receipts, runs - with the tensor KV nested inside the same map. The DAG capability tree is a view over this graph.
Qdrant	RAG - retrieval	Memories and facts, chunked and embedded, each with provenance. The research library in scope is research preprints and papers. Retrieval only.
HyperDX	Telemetry	Run traces, tool calls, reasoning traces, logs, and status - streamed per run, append-only, tied to the run's DAG nodes.

Rules that follow from the lock

- Do not collapse the three stores into one, and do not revive an earlier single-store migration plan without an explicit new ruling.
- N3 itself is strictly storage over SSD, accessed over a port. No GPU space, no CPU-resident full index, no inference or training compute inside the store.
- Cut over to a new store only after retrieval parity is shown. Parity first, cutover second.
- Deletions go to trash or backup first - recoverable - unless a permanent deletion is explicitly ordered for a named list.

5 Execution is not storage

core_lm is the execution plane. SurrealDB is persistence. One does not replace the other.

- **Execution plane (core_lm)** - runs the forward pass, holds the runtime, does the retrieval geometry at run time. A database does none of that.
- **Persistence (SurrealDB)** - holds the graph authority durably so runs, atoms, and receipts survive the session.
- SurrealDB therefore replaces loose file stores behind the mind graph - not the execution plane.

6 The typed workflow, step by step

Every run follows the same typed path. A step that cannot produce its output type does not pass to the next step.

1. **Mission in - type: Mission**
   Scope, direction, and goal stated and documented before work starts. If the expected output shape is not clear in one line - count, location, form - the run does not start.

2. **Nested store loads - type: Context**
   In N3 order: TreeSitter, Top MCP, Plugins, Skills.md, Hooks and Tooling. Hooks, tails, and port run before chat completion. Loaded once, kept live, routed by task.

3. **Packet formed - type: Packet**
   The mission becomes a packet: task, inputs, constraints, acceptance bar, and the pre-registered quality gate that will judge it.

4. **Retrieval - type: Evidence**
   Qdrant returns memories and facts with provenance. Evidence is attached to the packet, never merged silently into it.

5. **Lane executes - type: Run**
   A builder lane works the packet through tools, skills, and models. Every tool call and reasoning trace is captured in real time to HyperDX, per run, append-only.

6. **DAG updates - type: DAG Node**
   The run's steps become nodes in the SurrealDB graph as they happen - queued, running, verified, or blocked with a named reason - so the run can be watched, not just reported afterwards.

6 The typed workflow - continued

7. **Judge decides - type: Verdict**
   A judge checks the run against the pre-registered bar. Passed or blocked, with the reason named. The atom-intake gate and the prompt-adoption gate are separate gates with separate evidence - one gate's result is never cited as the other's.

8. **Receipt written - type: Receipt**
   Every run leaves a receipt in SurrealDB: what ran, inputs, outputs, verdict, and - on failure - the why-receipt. A run without a receipt did not happen, for system purposes.

9. **Atom admitted - type: Atom**
   Exactly one atom per passed run enters the mind map. Confabulations do not crystallise. Held items stay held, logged for review, until released through the gate.

10. **Capability tree updates - type: Capability**
    If the atom proves a capability, the DAG capability tree gains or strengthens that node - with its receipt and its HyperDX run linked. Otherwise the tree does not change.

Types at a glance

TYPE	MEANING
`Mission`	The ordered outcome, with scope and output shape stated.
`Packet`	A mission made workable: task, inputs, constraints, acceptance bar.
`Evidence`	Retrieved facts and memories, each with provenance, from Qdrant.
`Run`	One execution by a lane, fully captured to HyperDX.
`Verdict`	A judge's pass or block against the pre-registered bar, with reason.
`Receipt`	The durable record of the run and its verdict, in SurrealDB.
`Atom`	One proven unit admitted to the mind map - one per passed run.
`Capability`	A DAG node the system can prove it can do, backed by atom + receipt + run.

7 DAG capability tree

The capability tree is the DAG in SurrealDB, rendered by capability. It is not a separate file and not a drawn hierarchy.

- **Root** - the multi mind map itself, tensor KV nested in it.
- **Nodes** - capabilities, each one a judged atom. Cut by capability first: what the system can prove it can do.
- **Edges** - real graph edges: `depends_on`, `contains`, `parent_of`. Not a file-based adjacency list.
- **Leaves** - the tools, skills, and lanes that actually execute a capability, with receipts attached.
- **Provenance feed** - Qdrant supplies the memories and facts behind a node, so it can show why it claims the capability.
- **Proof feed** - HyperDX supplies whether that capability actually ran, on hardware, and with what result.

> **The gate:** if a capability has no judged atom, no receipt, and no run in HyperDX, it does not appear in the tree. Basin is a second view over the same DAG if needed. Tool and lane are filters, never the primary tree.

8 Gates, judges, and evidence

- **No synthetic runs ship, ever.** A synthetic or simulated run is not ship evidence. Proof runs run on hardware.
- **One atom per passed run,** through the judge gate. Nothing bypasses it.
- **Landed means live** in the running app - verified against the running process, its routes, and its interface - not a file on disk.
- **Every claim carries its evidence.** Verified, reported, inferred, or proposed - labelled as what it is, and withdrawn fast when wrong.
- **Observability is part of the run.** A silent run, or a failure with no why, is itself a failure.

9 Build order for the final run

Stores first. That is the whole correction, applied.

1. **Stand up the three stores.** SurrealDB for the mind graph, Qdrant for RAG with provenance, HyperDX for telemetry. Each on its own job, each reachable and checked live.
2. **Load the redone map into SurrealDB.** The map that was gone through by hand is the baseline - not the scattered files it replaced.
3. **Ingest the library straight.** Chunk and embed research material into Qdrant, with provenance on every chunk. No detours.
4. **Wire the typed workflow.** Mission to packet to lane to judge to receipt to atom, with N3 loading in its set order and capture streaming to HyperDX from the first run.
5. **Build the capability tree from the DAG.** Only nodes that pass the gate appear. Prune anything that cannot show atom + receipt + run.
6. **Run one proof run on hardware.** One real mission, end to end, observed live, receipt written, atom admitted or blocked with a named reason. That run - not a summary of it - is the evidence the system works.
7. **Spend the heat where it proves something.** Hard reasoning models go on the capability tree, the map, and the proof run - not on bulk work that free or local lanes can do.

> **Done means:** the three stores are live on their jobs, the workflow above runs end to end on hardware, the capability tree contains only proven nodes, and every claim in the master README points to a section, a receipt, or a run that backs it.

Master README maintenance: when a section changes, change its section file first, then update its anchor line in `README.md`. The table of contents on pages 2-3 is the index - if a path there does not resolve, the document is broken, not the reader.

10 Full forward pass - the whole pipeline, in order

One run, end to end. No step is skipped, and no step runs out of order to make a result look better.

1. MISSION	2. PARSE	3. STORE LOAD	4. RETRIEVE	5. BUILD

↓

6. ATTEND	7. EXECUTE	8. JUDGE	9. RECEIPT	10. ATOM

Phase A - Intake and structure

- ☐ **Mission typed in.** Scope, direction, goal, and output shape stated in one line before any work. LOCKED
- ☐ **TreeSitter parses first.** Code, docs, and configs become a structural graph - symbols, calls, imports - before any retrieval or attention. See §11. LOCKED
- ☐ **N3 loads in order.** TreeSitter, then Top MCP, then Plugins, then Skills.md, then Hooks and Tooling. Hooks, tails, and port run before chat completion. Loaded once, kept live. LOCKED
- ☐ **Packet formed.** Task, inputs, constraints, acceptance bar, and the pre-registered judge gate - all typed, all in the packet.

Phase B - Retrieve only what the packet needs

- ☐ **Query built from the parse, not from raw text alone.** Symbols and edges from TreeSitter narrow what Qdrant and SurrealDB are asked for.
- ☐ **Qdrant returns Evidence with provenance.** Memories and facts, each with its source. No provenance, no use.
- ☐ **SurrealDB returns the relevant subgraph.** Atoms, capabilities, and edges around the packet - not the whole map.
- ☐ **Nothing is merged silently.** Evidence stays labelled as Evidence until a judge admits it.

10 Full forward pass - continued

Phase C - Build the active set (not the whole model, not the whole map)

- [ ] **Attention selects.** Only the atoms, KV entries, tools, and skills the packet actually needs are activated. See §15. `TARGET`
- [ ] **Representation is built for this run only.** A working representation of the needed subgraph, in the run, for the run - then discarded or receipted, not hoarded.
- [ ] **TPE scoring applied where it belongs.** Toroidal coherence weights which atoms are selected. See §14 for its exact, narrow verified form. `VERIFIED (narrow)`
- [ ] **Brute force is cut here.** No full-map scan, no full-context stuffing, no activating every capability to answer one packet.

Phase D - Execute, judge, admit

- [ ] **Lane executes through tools and skills.** Every tool call and reasoning trace streams to HyperDX, per run, append-only, tied to the run's DAG nodes.
- [ ] **DAG updates live in SurrealDB.** Each step: queued, running, VERIFIED, or BLOCKED with a named reason.
- [ ] **Judge decides against the pre-set bar.** Pass or block, reason named. Intake gate and prompt-adoption gate stay separate.
- [ ] **Receipt written to SurrealDB.** What ran, inputs, outputs, verdict - and on failure, the why-receipt: command, exit, last output, expected result, evidenced cause, unblocker.
- [ ] **Exactly one atom admitted per passed run.** Blocked runs admit nothing. Held items stay held until released through the gate.
- [ ] **Capability tree updates only from that atom.** No atom, no new node, no rank change. See §12.

> **The gate for this whole section:** one full pass, on hardware, observed live, receipt written, atom admitted or blocked with a named reason. A pass described in prose - including this page - is not a pass that ran. `GATE`

11 Where TreeSitter sits

First. Before retrieval, before attention, before completion. If it is not parsed, it is not understood - it is only read.

Its exact place

POSITION	DETAIL
In N3 order	Position 1 of 5: TreeSitter, then Top MCP, Plugins, Skills.md, Hooks and Tooling. `LOCKED`
In the forward pass	Phase A, step 2 - immediately after the Mission is typed, before the Packet is finalised and before any store is queried.
Before completion	With the rest of N3: hooks, tails, and port run before the chat completion step. TreeSitter is part of that pre-completion load, never an afterthought inside generation.
In the stores	Its output is structure, and structure belongs in the graph: parse results become nodes and edges in SurrealDB, queryable like any other part of the map.

Checklist - what TreeSitter must do

- ☐ **Parse every code and structured input to the packet** - symbols, definitions, call sites, imports, and hierarchy - not a text skim.
- ☐ **Emit a structural graph, typed,** that the Packet, the retrieval query, and the DAG can all consume without re-parsing.
- ☐ **Narrow retrieval.** Retrieval asks for the symbols and relations the parse found, so Qdrant and SurrealDB return a subgraph, not a dump.
- ☐ **Feed capability detection.** What a codebase can do is read from its structure first; capability claims that contradict the parse do not pass.
- ☐ **Fail loud.** Unparseable input is BLOCKED with a named reason in the DAG - never silently treated as plain text and pushed on.
- ☐ **Stay a parser.** TreeSitter holds no weights, runs no inference, and stores nothing long-term itself. Structure in, graph out.

12 Capabilities - how they form and how they rank

A capability is not declared. It forms from proof, one judged run at a time - and it ranks only on evidence.

How a capability forms - checklist

- ☐ **A run passes its judge** against a pre-registered bar, on hardware.
- ☐ **A receipt is written** for that run in SurrealDB, and the run exists in HyperDX.
- ☐ **Exactly one atom is admitted** to the map from that run.
- ☐ **The atom becomes - or strengthens - a capability node** in the SurrealDB DAG, with real edges: `depends_on`, `contains`, `parent_of`.
- ☐ **Leaves attach:** the tools, skills, and lanes that actually executed it, with their receipts.
- ☐ **Provenance attaches:** the Qdrant memories and facts behind the claim, so the node can show why it claims it.

How capabilities rank - in this order

#	RANK SIGNAL	RULE
1	Proof strength	Hardware run with receipt outranks everything. Synthetic or simulated proof counts as zero and excludes the node.
2	Repeatability	Passed more than once, under its judge, outranks passed once. A single pass is a claim, not a standing.
3	Judge record	Pass record against pre-set bars - not self-report. Blocks and their reasons stay visible in the record.
4	Provenance completeness	Every supporting fact traceable in Qdrant outranks a node with gaps in its why.
5	Runtime cost	From HyperDX: latency, tokens, and energy per successful run. Same proof, lower dissipation ranks higher. See §14.
6	Dependence	How much of the DAG depends on this node. Central and proven outranks central and merely claimed - claimed does not rank at all.
7	Toroidal coherence	Coherence weighting may order atom selection inside a run. It is one signal in selection - never a substitute for signals 1-4. **`VERIFIED (narrow)`**

Decay rule: a capability that is not re-proven does not keep its rank forever. Rank is a current standing, re-earned - not a title granted once.

13 Distillation - removing the weights to the map

Teachers are distilled into the map. The map is what the system keeps - not a shelf of separate teacher weights it has to carry and run.

One unified mind map, from varying open-source models, specialised by distillation into that single map. Not one map per teacher. The model that works from the map is the model; a teacher is a source, never the runtime. `TARGET`

Checklist - one teacher, distilled

- ☐ **Teacher named and scoped.** Which model, which domain or basin, and what it is supposed to teach the map - stated before capture starts.
- ☐ **Every run captured in full.** Tool calls and reasoning traces, streaming, append-only, per run, tied to DAG nodes. Capture is not admission - it is raw material only.
- ☐ **Falsifier applied.** Captured material is checked for confabulation before it can move. Confabulations do not crystallise.
- ☐ **Frontier / judge approval before crystallising.** New material is logged, run, and approved through the gate first - otherwise the map gets trained on its own noise.
- ☐ **Crystallisation hold respected.** Held episodes stay held, logged for review, until released. Nothing slips in because a run felt good.
- ☐ **Distilled into atoms, edges, and nested tensor KV** in the SurrealDB map - the proven content becomes map structure, in the map's own form.
- ☐ **Teacher weights then removed from the runtime path.** The system answers from the map and its own model. If removing the teacher breaks the capability, the distillation did not happen - it was borrowing.
- ☐ **Proven on unseen problems.** The distilled capability must derive an answer to a problem with no stored trajectory to copy. Retrieval of a solved example is not proof of distillation.
- ☐ **Receipt and rank updated.** The capability node records its teacher source, its proof run, and its new rank under §12.

> **Failure mode this prevents:** five teachers, five weight sets, five behaviours, and no single map anyone can inspect. Distillation is how the system stays one mind - teachers come and go, the map remains.

14 TPE, toroidal scoring, and thermodynamic dissipation

The objective is plain: do the work with less waste. Less compute touched, less KV moved, less energy burned per proven result.

What is actually verified - and what is not

- **`VERIFIED (narrow)`** A toroidal coherence score - $\cos(\Delta\theta_f) + \cos(\Delta\theta_s)$ - weighting atom selection. That is the in-repo evidence. It is a scoring function, not a transformer attention layer, and there is no end-to-end benchmark replacing standard position handling with it.
- **`VERIFIED (narrow)`** TPE and related kernels exist as bench-crate kernels, and the gain form is closed on disk as $(\text{coherence} + 2)^{1.5}$ in log-domain form - frozen until a separately pre-registered fit replaces it. Receipts expose the gain and its formula. A bench kernel is not a forward pass.
- **`TARGET`** TPE inside the live forward pass, reducing dissipation at runtime. That is the design aim below - it must be proven end to end, on hardware, before it is claimed.

Checklist - TPE in the pass

- ☐ **Coherence computed per candidate atom,** from the toroidal score above - nothing wider is claimed for it.
- ☐ **Gain applied as frozen:** $(\text{coherence} + 2)^{1.5}$, log-domain. No silent retuning. A new fit is pre-registered, separately, or it does not ship.
- ☐ **Gain and formula written to the receipt** for every run that used them, so any selection can be replayed and checked.
- ☐ **Selection effect measured:** atoms selected with TPE weighting against atoms selected without it, same packet, judged the same way.

Checklist - reducing thermodynamic dissipation

- ☐ **Measure the waste, per run, in HyperDX:** tokens processed, KV entries touched, map nodes visited, tool calls made, wall time, and energy where the hardware reports it.
- ☐ **Set the unit:** dissipation per proven result - per passed, receipted run - not per token generated. Fast waste is still waste.
- ☐ **Cut by not building:** every subsystem not activated for a packet is dissipation avoided at source. See §15.
- ☐ **Compare honestly:** active build against brute-force build, same packet, same judge. The saving is the measured difference, or there is no saving to claim.

15 Active inference at runtime - build only what is needed

The runtime does two things - attention and representation - and it does them for this packet, at this moment. It does not run the whole machine to answer one question.

The principle

Brute-force forward-pass logic activates everything and lets the answer fall out. This system does the opposite: it works out what the packet needs, builds only that - the retrieval set, the working representation, the tools, the execution path - and runs only that. Everything not built is compute not spent and dissipation not incurred.

Checklist - one runtime pass

- ☐ **Need is stated from the Packet and the parse.** What must be retrieved, what must be executed, and what would prove it - before anything is activated.
- ☐ **Attention selects the active set:** atoms, nested tensor KV entries, Evidence, tools, and skills for this packet only. Selection is receipted, so it can be audited.
- ☐ **Representation is built from the active set only** - a working model of the relevant subgraph, sized to the task, not to the whole map.
- ☐ **Surprise drives the next step.** Where the representation does not match what execution finds, the active set is revised - retrieve more, or execute differently - rather than pushing on with a wrong build.
- ☐ **Execution runs only the built path.** Tools and lanes outside the active set stay cold. No speculative full runs.
- ☐ **The build is torn down or receipted.** Working representations do not silently become map truth. Only a judged, receipted atom changes the map (§6, §12).
- ☐ **Dissipation is read back from HyperDX** and compared with the brute-force baseline for the same packet (§14). The comparison is part of the run's record.

Whole-system sign-off - the whole thing, in one list

- ☐ Stores live on their jobs: SurrealDB, Qdrant, HyperDX (§4).
- ☐ Forward pass runs §10 in order, TreeSitter first in N3 (§11), on hardware.
- ☐ Every capability in the tree is formed and ranked under §12 - no exceptions.
- ☐ Every teacher in the system was distilled under §13, weights out of the runtime path.
- ☐ TPE is used only in its verified form, gain frozen, until an end-to-end proof widens it (§14).
- ☐ Runtime builds only what each packet needs, and the saving is measured, not asserted (§15).